

A Comparative Deep Learning Framework for PM_{2.5} Air Quality Forecasting Using LSTM, RNN, and GRU

Abstract—Air pollution is a major environmental concern that requires reliable monitoring and forecasting techniques. Among various air pollutants, PM_{2.5} is an important indicator of air quality because it represents fine particulate matter present in the atmosphere. This paper presents a comparative deep learning framework for PM_{2.5} air-quality forecasting using Long Short-Term Memory (LSTM), Recurrent Neural Network (RNN), and Gated Recurrent Unit (GRU) models. The study uses the PRSA Aotizhongxin air-quality dataset, which contains hourly air-quality and meteorological observations from March 2013 to February 2017. The dataset is preprocessed by handling missing values, constructing a datetime index, selecting relevant features, and applying Min-Max scaling. A chronological 80%–20% train-test split is used to preserve temporal order. A 24-hour sliding window is then used to generate sequential input samples. LSTM, RNN, and GRU models are trained under identical configurations. Experimental results show that the GRU model achieves the best performance with MAE of 18.14 $\mu\text{g}/\text{m}^3$, RMSE of 32.04 $\mu\text{g}/\text{m}^3$, and R^2 of 0.8527, followed closely by LSTM and RNN. A prototype web application is also developed to demonstrate real-time inference. The study identifies the comparatively best-performing recurrent architecture and demonstrates the applicability of deep learning models for PM_{2.5} time-series forecasting.

Index Terms—PM_{2.5}, Air Quality Forecasting, Deep Learning, LSTM, RNN, GRU, Time-Series Forecasting, Air Pollution

I. INTRODUCTION

A. Background

Air pollution is one of the major environmental challenges affecting air quality and human well-being. The concentration of air pollutants changes continuously over time because of various environmental, atmospheric, and meteorological conditions. Therefore, accurate air-quality forecasting is important for monitoring pollution levels and understanding future airquality conditions.

PM_{2.5} is one of the most important indicators of air pollution. It represents fine particulate matter with a diameter of 2.5 micrometers or smaller. Because of its small size, PM_{2.5} can penetrate deeply into the respiratory system. Therefore, forecasting PM_{2.5} concentration is an important task in airquality analysis.

Air-quality data generally contain sequential observations collected at different time intervals. Deep learning techniques, particularly recurrent neural networks, are suitable for this type of sequential data because they can learn relationships between observations over time.

In this work, three recurrent deep learning models, namely LSTM, RNN, and GRU, are implemented and compared for PM_{2.5} forecasting. The models use historical air-quality and meteorological features to predict the next PM_{2.5} value.

B. Problem Statement

PM_{2.5} concentration varies over time and is influenced by multiple environmental and meteorological factors. Predicting future PM_{2.5} values from historical observations is therefore a challenging time-series forecasting problem.

The main problem addressed in this work is to determine how effectively LSTM, RNN, and GRU models can learn temporal patterns from historical air-quality data and forecast future PM_{2.5} concentrations under the same preprocessing and training conditions.

C. Motivation

The motivation of this work comes from the need for reliable air-quality forecasting using historical environmental data. The specific motivating factors are:

- The importance of monitoring air pollution.
- The importance of PM_{2.5} as an air-quality indicator.
- The sequential nature of air-quality data.
- The ability of recurrent deep learning models to process time-series data.
- The need to compare different recurrent architectures under a common experimental framework.

D. Objectives

The main objective is to develop and compare deep learning models for PM_{2.5} air-quality forecasting. The specific objectives are:

- 1) To load and inspect the air-quality dataset.
- 2) To identify and handle missing values.
- 3) To construct a proper datetime index.
- 4) To select relevant air-quality and meteorological features.
- 5) To normalize the input features using Min-Max scaling.
- 6) To divide the dataset chronologically into training and testing sets.
- 7) To construct 24-hour sequential input data.
- 8) To develop LSTM, RNN, and GRU models.
- 9) To train the models using a consistent configuration.
- 10) To evaluate the models using MAE, RMSE, and R^2 Score.
- 11) To compare the overall forecasting performance of the three models.
- 12) To develop a prototype web application for interactive inference.

E. Contributions

The major contributions of this work are as follows:

- Development of a complete preprocessing pipeline for hourly air-quality data.
- Construction of 24-hour time-series sequences.
- Implementation of LSTM, RNN, and GRU forecasting models under identical conditions.
- Evaluation using MAE, RMSE, and R^2 Score.
- Visualization of training behavior and prediction performance.
- Comparative identification of the better-performing deep learning model.
- Development of a prototype web application (AirSense AI) for real-time PM_{2.5} inference.

II. RELATED WORK

A. PM_{2.5} Air Quality Forecasting

PM_{2.5} forecasting has been studied using statistical methods, machine learning techniques, and deep learning architectures. Traditional forecasting techniques can model temporal relationships, but complex air-quality data may contain nonlinear relationships among pollutants and meteorological variables. This has encouraged researchers to explore machine learning and deep learning approaches for air-quality prediction [1], [4], [5].

B. Deep Learning for Time-Series Forecasting

Deep learning models are capable of learning complex nonlinear relationships from data. For time-series forecasting, recurrent neural network architectures are particularly useful because they process data sequentially and can learn temporal dependencies [2], [3].

C. Recurrent Neural Network (RNN)

Recurrent Neural Network (RNN) is a neural network architecture designed for sequential data. Unlike traditional feed-forward neural networks, RNNs maintain information from previous time steps through recurrent connections. However, standard RNNs may have difficulty learning long-term dependencies in long sequences.

D. Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) is an advanced form of recurrent neural network designed to handle long-term dependencies. LSTM uses a memory mechanism and gating operations to control the flow of information through the network [2].

E. Gated Recurrent Unit (GRU)

Gated Recurrent Unit (GRU) is a gated recurrent architecture designed to learn temporal dependencies while using a relatively simpler structure than LSTM [3]. GRU uses gating mechanisms to control information flow and can be applied effectively to sequential forecasting problems.

F. Research Gap

Although different deep learning models have been applied to time-series and air-quality forecasting, their performance can vary depending on the dataset, preprocessing approach, sequence length, and training configuration [6]–[9]. Therefore, this work focuses on a controlled comparative evaluation of RNN, LSTM, and GRU using the same dataset, preprocessing pipeline, 24-hour sequence length, and basic training configuration.

III. DATASET AND DATA DESCRIPTION

A. Dataset Description

The work uses the PRSA Aotizhongxin air-quality dataset [1]. The dataset contains hourly air-quality and meteorological observations from the Aotizhongxin monitoring station covering the period from March 1, 2013 to February 28, 2017. The dataset file used is PRSA_Data_Aotizhongxin_20130301-20170228.csv.

B. Feature Description

The selected features used in this work are shown in Table I.

TABLE I
SELECTED FEATURES

Feature	Description
PM _{2.5}	Fine particulate matter concentration
PM ₁₀	Particulate matter concentration
SO ₂	Sulfur dioxide concentration
NO ₂	Nitrogen dioxide concentration
CO	Carbon monoxide concentration
O ₃	Ozone concentration
TEMP	Temperature
PRES	Atmospheric pressure
DEWP	Dew point
RAIN	Rainfall
WSPM	Wind speed

C. Target Variable

The target variable is PM_{2.5}. The remaining selected environmental and meteorological variables are used as input features. The forecasting structure can be represented as:

Historical Environmental Features → Deep Learning Model → Future PM

(1)

D. Dataset Statistics

Table II summarizes the key characteristics of the dataset.

TABLE II
DATASET CHARACTERISTICS

Property	Value
----------	-------

Dataset	PRSA Aotizhongxin
Data Type	Time-Series
Observation Frequency	Hourly
Time Period	2013–2017
Target Variable	PM _{2.5}
Selected Features	11
Input Features	10
Sequence Length	24 Hours
Training Data	80%
Testing Data	20%

IV. METHODOLOGY

A. Proposed Framework

The proposed framework consists of several stages: dataset loading, data inspection, missing-value handling, datetime processing, feature selection, feature scaling, chronological traintest splitting, sequence generation, model training, prediction, and evaluation.

The overall workflow is:

Raw Dataset → Data Preprocessing → Feature Scaling
→ Train-Test Split → 24-Hour Sequence Generation
→ LSTM/RNN/GRU → PM_{2.5} Prediction → Evaluation

(2)

B. Data Preprocessing

- 1) Missing Value Handling: Numerical variables are handled using median imputation, while categorical variables are handled using mode imputation.
- 2) Date-Time Processing: Year, month, day, and hour attributes are combined into a single datetime index.
- 3) Data Sorting: Observations are sorted chronologically.
- 4) Feature Selection: Ten environmental and meteorological variables are used as input features, with PM_{2.5} as the target.
- 5) Target Variable Separation: The input matrix is represented by X and the target by y .

C. Feature Scaling

The input features are normalized using the Min-Max Scaler:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (3)$$

Feature scaling is applied because the selected input variables have different numerical ranges.

D. Chronological Train-Test Splitting

The dataset is divided into 80% training data and 20% testing data while preserving chronological order. This approach is appropriate for time-series forecasting because future observations should not be used during training.

E. Time-Series Sequence

Generation A sequence length of 24 is used:

$$\text{TIME STEPS} = 24 \quad (4)$$

This means that the previous 24 hourly observations are used as input to predict the next PM_{2.5} value.

F. Model Architectures

All three models share the same high-level structure: a recurrent layer with 64 units, a dropout layer (rate 0.2), and a dense output layer with one neuron.

- LSTM: Input → LSTM(64) → Dropout(0.2) → Dense(1)
- RNN: Input → SimpleRNN(64) → Dropout(0.2) → Dense(1)
- GRU: Input → GRU(64) → Dropout(0.2) → Dense(1)

G. Model Training Configuration

The three models are trained using the configuration shown in Table III.

TABLE III MODEL TRAINING CONFIGURATION	
Parameter	Value
Sequence Length	24
Hidden Units	64
Dropout	0.2
Optimizer	Adam
Loss Function	MSE
Epochs	20
Batch Size	32
Validation Split	20%
Output Neurons	1

H. Model Evaluation Metrics

- Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (5)$$

- Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (6)$$

- R^2 Score:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (7)$$

V. EXPERIMENTAL SETUP

A. Experimental Environment

The experiments are implemented using Python in a Google Colab environment.

B. Software and Libraries

The following Python libraries are used (Table IV).

Library	Purpose
Pandas	Data loading and manipulation
NumPy	Numerical operations
Matplotlib	Visualization
Scikit-learn	Preprocessing and evaluation
TensorFlow/Keras	Deep learning models
Joblib	Saving the scaler

VI. EXPERIMENTAL RESULTS AND ANALYSIS

A. Quantitative Metrics

Table V summarizes the performance of the three models on the held-out test set.

Model	MAE ($\mu\text{g}/\text{m}^3$) ↓	RMSE ($\mu\text{g}/\text{m}^3$) ↓	R^2 ↑
LSTM	18.42	32.30	0.8503
RNN	18.95	32.84	0.8453
GRU	18.14	32.04	0.8527

The GRU model attains the lowest error metrics and the highest coefficient of determination, followed closely by LSTM. The Simple RNN exhibits slightly higher errors, which is consistent with its known limitations in learning long-term dependencies.

B. Training and Validation Loss Analysis

Figures 2–4 illustrate the training and validation loss curves. All models converge stably within 20 epochs. No severe overfitting is observed.

C. Actual vs. Predicted Analysis

Figure 1 shows a representative segment of actual versus predicted $\text{PM}_{2.5}$ values. The forecasts closely track both the overall trend and the short-term fluctuations.

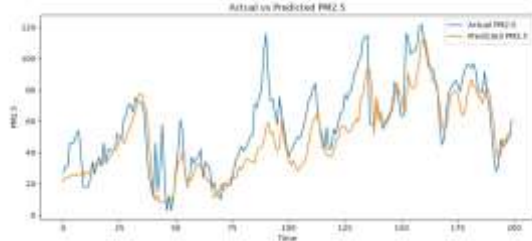


Fig. 1. Actual versus predicted $\text{PM}_{2.5}$ concentrations (first 200 test samples).

D. Best Model Identification

Based on the experimental results, the GRU model achieved the best overall forecasting performance with:

- MAE: $18.14 \mu\text{g}/\text{m}^3$
- RMSE: $32.04 \mu\text{g}/\text{m}^3$

- R^2 Score: 0.8527

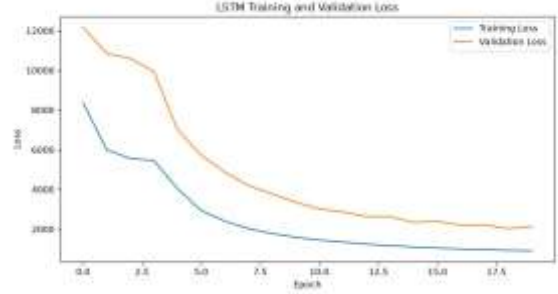


Fig. 2. LSTM training and validation loss.

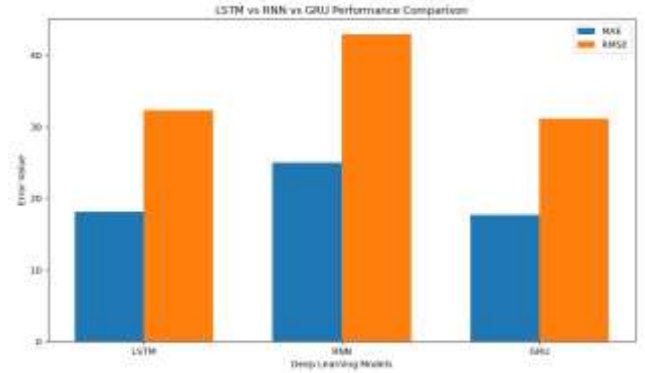


Fig. 3. RNN training and validation loss.

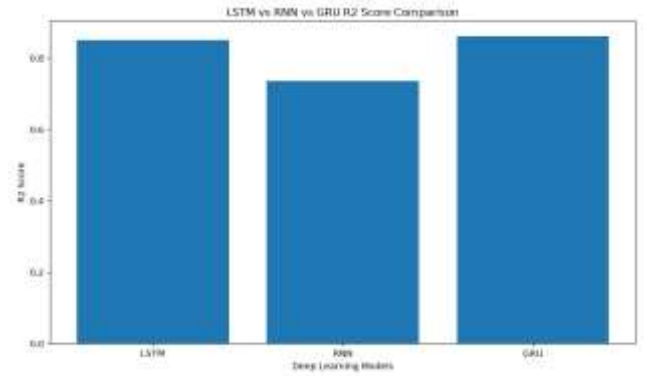


Fig. 4. GRU training and validation loss.

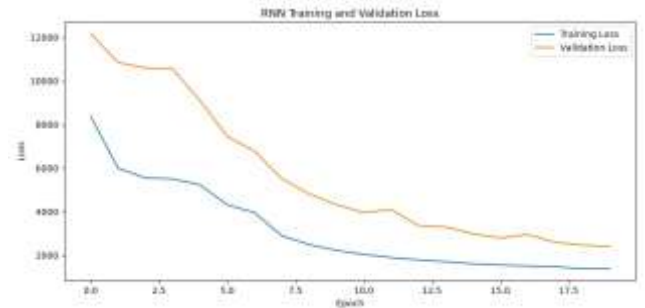


Fig. 5. MAE and RMSE comparison among LSTM, RNN, and GRU.

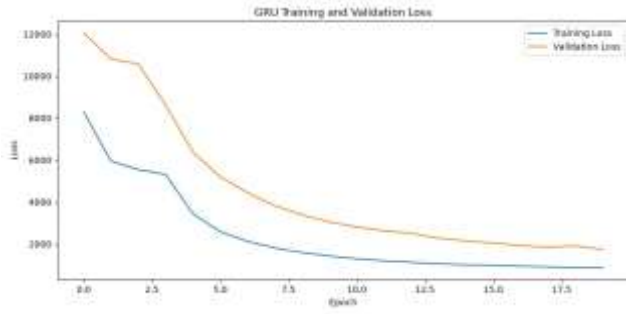


Fig. 6. R^2 score comparison among the three models.

VII. SYSTEM PROTOTYPE: AirSense AI WEB APPLICATION

To demonstrate practical usability of the trained models, a lightweight web-based prototype named AirSense AI was developed. The application allows users to select a model (LSTM / RNN / GRU), input recent meteorological and pollutant observations, obtain a $PM_{2.5}$ forecast, and visualize historical trends together with model performance metrics.

Figures 7–10 show representative screenshots of the deployed interface.



Fig. 7. AirSense AI main dashboard interface.

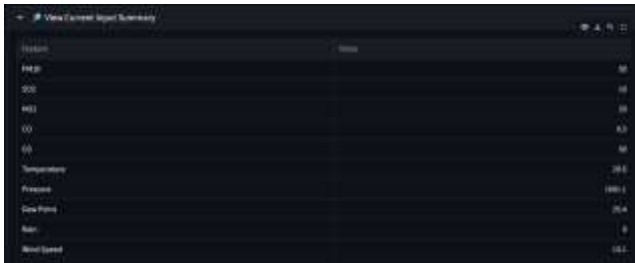


Fig. 8. Model selection and input feature panel.

The prototype integrates the saved Keras models (.h5) and the Min-Max scaler, enabling end-to-end inference without retraining.



Fig. 9. Forecast result visualization.



Fig. 10. Historical trend and performance comparison view.

VIII. DISCUSSION

The experimental results confirm that gated recurrent architectures (GRU and LSTM) outperform the vanilla RNN for $PM_{2.5}$ forecasting on this dataset. The modest advantage of GRU over LSTM can be attributed to its simpler gating mechanism, which reduces the number of parameters and may improve generalization under the chosen hyper-parameter regime.

All models achieve R^2 values above 0.84, indicating that more than 84% of the variance in hourly $PM_{2.5}$ is explained by the multi-variate meteorological and pollutant features. The developed web application further validates the practical applicability of the framework.

IX. LIMITATIONS

- The experiment is conducted using a single monitoring station (Aotizhongxin).
- Only three recurrent architectures (RNN, LSTM, GRU) are evaluated.
- A fixed set of hyperparameters is used; systematic hyperparameter optimization is not performed.
- Only a 24-hour historical input window is used.
- External factors such as traffic density and regional pollutant transport are not incorporated.

X. CONCLUSION AND FUTURE WORK

A. Conclusion

This paper presented a comparative deep learning framework for $PM_{2.5}$ air-quality forecasting using LSTM, RNN, and GRU models. On the Beijing Aotizhongxin

dataset, the GRU model delivered the best performance (MAE = 18.14, RMSE = 32.04, $R^2 = 0.8527$). A prototype web application was also implemented to support interactive inference. The study provides a clean, reproducible baseline and demonstrates the practical utility of recurrent neural networks for short-term air-quality prediction.

B. Future Work

Future research directions include:

- 1) Hyperparameter optimization (hidden units, dropout, learning rate, sequence length).
- 2) Evaluation of different historical windows (12, 48, 72 hours).
- 3) Advanced architectures such as Bidirectional LSTM, CNN-LSTM, Attention-based networks, and Transformers.
- 4) Multi-step PM_{2.5} forecasting (next 6, 12, or 24 hours).
- 5) Incorporation of Explainable AI techniques (e.g., SHAP).
- 6) Comparison with traditional machine learning algorithms (Random Forest, XGBoost).
- 7) Multi-location evaluation across multiple monitoring stations.

ACKNOWLEDGMENT

The authors thank the providers of the Beijing Multi-Site Air Quality dataset for making the measurements publicly available.

REFERENCES

- [1] S. Chen, "Beijing Multi-Site Air Quality," UCI Machine Learning Repository, 2017, doi: 10.24432/C5RK5G.
- [2] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/NECO.1997.9.8.1735.
- [3] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar, 2014, pp. 1724–1734, doi: 10.3115/v1/D14-1179.
- [4] R. Yan, J. Liao, J. Yang, W. Sun, M. Nong, and F. Li, "Multihour and multi-site air quality index forecasting in Beijing using CNN, LSTM, CNN-LSTM, and spatiotemporal clustering," *Expert Systems with Applications*, vol. 169, Art. no. 114513, 2021, doi: 10.1016/j.eswa.2020.114513.
- [5] J. Yang, R. Yan, M. Nong, J. Liao, W. Sun, and F. Li, "PM_{2.5} concentrations forecasting in Beijing through deep learning with different inputs, model structures and forecast time," *Atmospheric Pollution Research*, vol. 12, no. 9, Art. no. 101168, 2021, doi: 10.1016/j.apr.2021.101168.
- [6] J. Zhang and S. Li, "Air quality index forecast in Beijing based on CNNLSTM multi-model," *Chemosphere*, vol. 308, Art. no. 136180, 2022, doi: 10.1016/j.chemosphere.2022.136180.
- [7] B. Zhang, Y. Rong, R. Yong, D. Qin, M. Li, G. Zou, and J. Pan, "Deep learning for air pollutant concentration prediction: A review," *Atmospheric Environment*, vol. 290, Art. no. 119347, 2022, doi: 10.1016/j.atmosenv.2022.119347.
- [8] H. Liu, G. Yan, Z. Duan, and C. Chen, "Intelligent modeling strategies for forecasting air quality time series: A review," *Applied Soft Computing*, vol. 102, Art. no. 106957, 2021, doi: 10.1016/j.asoc.2020.106957.
- [9] A. Masood and K. Ahmad, "A review on emerging artificial intelligence (AI) techniques for air pollution forecasting: Fundamentals, application and performance," *Journal of Cleaner Production*, vol. 322, Art. no. 129072, 2021, doi: 10.1016/j.jclepro.2021.129072.
- [10] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, Savannah, GA, USA, 2016, pp. 265–283.
- [11] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011.
- [12] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.